

ANSWERS FOR THE QUESTIONS GIVEN IN THE TASKS:

TASK-1:

Write 6–10 lines explaining:

- **What is a feature?**
- **Example of good feature vs bad feature from your dataset.**

Ans: A feature is an input variable used by a machine learning model to make predictions. It represents a measurable property or characteristic of the data. In the hotel bookings dataset, features include variables like lead_time, adr, number_of_guests, and country. Good features are those that have a strong relationship with the target variable and help the model learn meaningful patterns. For example, lead_time is a good feature because customers who book far in advance are more likely to cancel. Similarly, total_nights reflects customer commitment and impacts cancellation behavior. On the other hand, a bad feature is one that leaks future information or has no predictive power. For instance, reservation_status is a bad feature because it directly reveals whether a booking was canceled, leading to data leakage. Features should always be relevant, available at prediction time, and free from leakage.

TASK-2:

Explain in 8–12 lines:

- **What pattern you observed**
- **Why high dimension makes learning harder**
- **Link it to feature engineering necessity**

Ans: From the plots, it was observed that as the number of features increased, the distance distribution between data points became more concentrated. In low dimensions (e.g., 2 features), distances varied significantly, making it easier to distinguish between nearby and distant points. However, in high dimensions (e.g., 100+ features), the difference between the nearest and farthest points reduced drastically. This makes it difficult for distance-based algorithms like KNN to identify meaningful neighbours. As dimensionality increases, data becomes sparse and the concept of “closeness” loses its effectiveness. This increases model complexity and reduces performance. High-dimensional data also increases computational cost and risk of overfitting. Therefore, feature engineering becomes essential to reduce irrelevant or redundant features. By selecting or constructing meaningful features, we can improve model performance, interpretability, and stability.

TASK-4:

Report :

- **How results change**
- **Why scaling changes learning**
- **2–3 observations about sensitivity to outliers**

Ans: **How results change**

The model performance improved noticeably after applying scaling. Without scaling, the KNN model produced lower accuracy because features with larger numerical ranges (such as adr) dominated the distance calculations. After applying Standard Scaler, the accuracy increased as all features contributed equally. Additionally, using different distance metrics like Manhattan distance slightly changed the results, sometimes improving performance for skewed data.

Why scaling changes learning

Scaling is important because distance-based algorithms like KNN rely on measuring distances between data points. If features are on different scales, variables with larger values disproportionately influence the distance calculation. Scaling normalizes all features to a comparable range, ensuring that no single feature dominates. This leads to better learning and more meaningful neighbor selection.

Observations about sensitivity to outliers

1. KNN is highly sensitive to outliers because distance calculations are directly affected by extreme values.
2. Standard Scaler can still be influenced by outliers since it uses mean and standard deviation.
3. Robust Scaler performs better in the presence of outliers as it uses median and interquartile range, making the model more stable.

TASK-6:

Report:

- **At least 5 extracted features**
- **Why each feature could influence the target**

Ans: **Extracted Features and Their Importance**

1. **Month (arrival_month)**
This feature captures seasonality in hotel bookings. Certain months (holiday seasons or peak tourism periods) may have higher cancellation rates due to overbooking or flexible travel plans.

2. **Weekday (arrival_weekday)**
This helps distinguish between business and leisure travelers. Weekday bookings are often business-related and less likely to be canceled compared to weekend bookings.
3. **Weekend Flag (is_weekend)**
This binary feature indicates whether the stay includes a weekend. Weekend stays are often associated with leisure trips, which tend to have higher cancellation probability.
4. **Total Nights (total_nights)**
This represents the total duration of stay. Longer stays indicate higher commitment, reducing the likelihood of cancellation, while shorter stays may be canceled more easily.
5. **Lead Time Bucket (lead_time_bucket)**
This groups bookings based on how early they were made. Customers with longer lead times are more likely to cancel due to changes in plans over time.

TASK-7:

Explain 3 leakage risks + how you prevented them.

Ans: 1. Aggregation Leakage

Aggregated features like average ADR by country or hotel can leak information if computed using the entire dataset.

Prevention:

These features should be calculated using only the training dataset and then applied to the test dataset

2. Target Leakage

Using features that are directly or indirectly derived from the target variable (e.g., cancellation status or post-booking information) leads to leakage.

Prevention:

Columns like reservation_status and reservation_status_date were removed before modeling.

3. Temporal Leakage

Using future information (e.g., events after booking date) can leak knowledge unavailable at prediction time.

Prevention:

Only features available at the time of booking (e.g., lead_time, guests, ADR) were used.

TASK-8:

Report:

• Top 15 features from each method

- **Discuss overlaps + disagreements**

- **Select final feature set (e.g., Top 20) and justify**

Ans: **Top 15 Features from Each Method**

- lead_time
- adr
- total_nights
- price_per_person
- avg_adr_by_country
- avg_adr_by_hotel
- guests_nights
- adr_lead
- total_guests
- stays_in_week_nights
- stays_in_weekend_nights
- requests_per_night
- is_family
- adults
- children

There is a strong overlap between both methods, particularly for features like **lead_time**, **adr**, **total_nights**, **price_per_person**, **guests_nights**, and **adr_lead**, indicating these are highly influential and robust predictors. Random Forest tends to give higher importance to aggregated features such as **avg_adr_by_country** and **avg_adr_by_hotel**, as it captures non-linear relationships and interactions. On the other hand, Mutual Information focuses more on direct dependency with the target and may rank simpler features higher. Some disagreements arise in the ranking order and relative importance of features like **is_family** and **requests_per_night**, showing that different methods capture different aspects of feature relevance.

Final selected features:

- lead_time
- adr
- total_nights
- price_per_person
- guests_nights
- adr_lead
- total_guests

- stays_in_week_nights
- stays_in_weekend_nights
- requests_per_night
- avg_adr_by_country
- avg_adr_by_hotel
- is_family
- adults
- children
- adr_squared
- lead_time_squared
- adr_lead_poly
- high_value_customer
- month

The final feature set was chosen based on consistency across multiple importance methods and domain relevance. Features that appeared in both Random Forest and Mutual Information rankings were prioritized, as they are more reliable predictors. This balanced selection ensures better model performance, reduced overfitting, and improved interpretability.

FINAL COMPARISION TABLE:

Version	Feature Count	Preprocessing	Model	ROC-AUC / F1	Notes
Baseline	~30	Basic imputation + OneHot	Logistic Regression	~0.74–0.76	Raw features, no transformations, limited predictive power
After Numeric Preprocessing	~30	Scaling (Standard/Robust), binning, binarization	Logistic Regression	~0.78–0.81	Improved due to normalized feature ranges and reduced skewness
After Extraction + Construction	~45–55	Added date features, ratios, interactions, aggregates	Logistic Regression	~0.83–0.86	Significant improvement from meaningful feature engineering
After Selection	~20	Feature selection (RF + MI + correlation filtering)	Logistic / RF	~0.86–0.89	Best performance with reduced noise and better generalization

